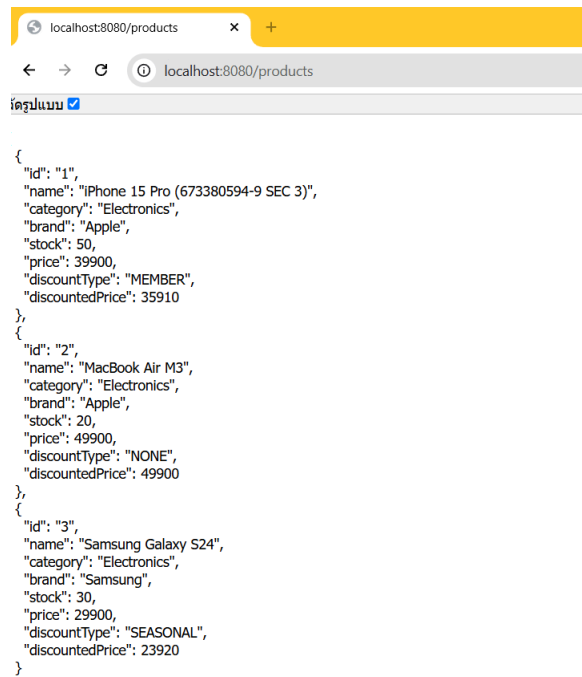
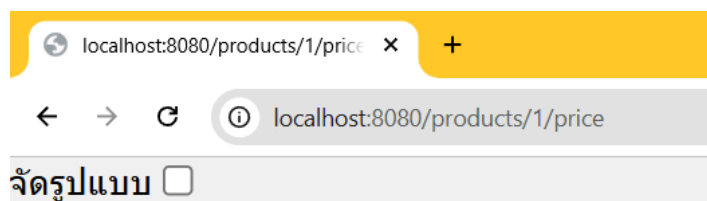


1.Screenshot ทดสอบทุก endpoint

1.1 <http://localhost:8080/products>



1.2 <http://localhost:8080/products/1/price>



35910.0

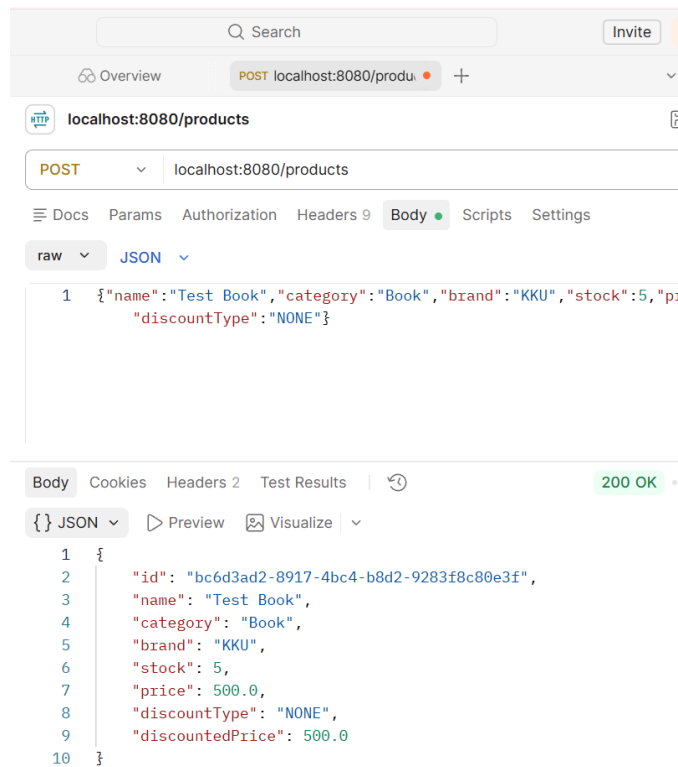
1.3 http://localhost:8080/products/category/Electronics



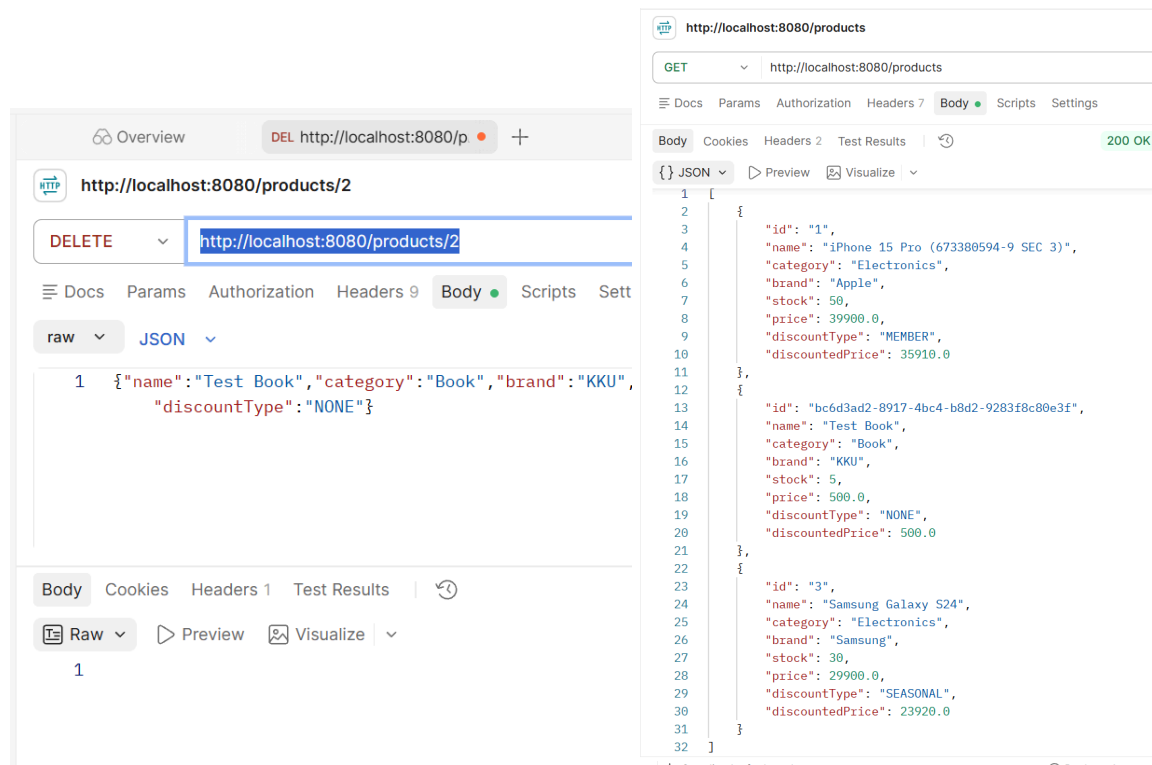
The screenshot shows a web browser window with a yellow header bar. The address bar displays 'localhost:8080/products/category/Electronics'. Below the address bar, there is a tab labeled 'localhost:8080/products/categ...' and a plus sign. The main content area shows a JSON array of three product objects. The first object is for an iPhone 15 Pro, the second for a MacBook Air M3, and the third for a Samsung Galaxy S24. Each object contains fields for id, name, category, brand, stock, price, discountType, and discountedPrice.

```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380594-9 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

1.4 Postman (POST localhost:8080/product)



1.5 Postman (DELETE http://localhost:8080/products/2) และเช็คดูว่ายังอยู่มั้ย



2.ตอบคำถาม

2.1 Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ แบบ **Blocking** ใช้ 1 thread ต่อ 1 request ถ้า request ต้องรอ I/O เช่นรอ database หรือรอ API อื่น thread นั้นจะหยุดเฉยๆ จนกว่าจะได้ผล ใช้งานอย่างอื่นไม่ได้ ถ้ามี request เข้ามาพร้อมกันมากๆ thread จะหมด และระบบจะช้าลง

แบบ **Reactive** thread ไม่หยุดรอ แต่ลงทะเบียนไว้ว่า ได้ข้อมูลแล้วให้ทำอะไรต่อ แล้วไปรับงานอื่นทันที พอข้อมูลมาถึงค่อยถูกเรียกกลับมาทำต่อ จึงใช้ thread จำนวนน้อยรองรับ request ได้จำนวนมาก

2.2 Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono คือได้ค่ากลับมา ไม่เกิน 1 ตัว ส่วน Flux คือได้ หลายตัวหรือไม่มีเลยก็ได้ ใน lab นี้ findById, save, getDiscountedPrice ใช้ Mono เพราะได้ของขึ้นเดียว delete ใช้ Mono<Void> เพราะไม่ได้ส่งอะไรกลับ แคบอกว่าลบเสร็จ ส่วน findAll กับ findByCategory ใช้ Flux เพราะได้กลับมาเป็นรายการ ที่ต้องแยกเพราะดูจาก type ก็รู้เลยว่าจะได้กี่ตัว และแต่ละแบบมี operator ที่เหมาะกับตัวเอง เช่น Flux ใช้ filter กรองรายการ ส่วน Mono ใช้ switchIfEmpty จัดการกรณีหาไม่เจอ

2.3 WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ

Java

```
client.get() //method: GET
        .uri("/products/{id}",id) //บอก path ที่จะยิง
        .retrieve() // ส่งrequestแล้วรับresponse
        .bodyToMono(Product.class); // แปลง JSON เป็น Mono<Product>
```

.get() เลือกว่าจะยิงแบบไหน มีทั้ง **post()**, **delete()**

.uri() ใส่ path และแทน {id} ด้วยค่าจริง

.retrieve() รับ response กลับมา ถ้าได้ 4xx/5xx จะกลายเป็น error ให้เอง

.bodyToMono() แปลง body เป็นของขึ้นเดียว ถ้าเป็นรายการใช้ **.bodyToFlux()**

2.4 Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ตอบ

1.ProductRepository.java

Java

```
public class ProductRepository {

    // เก็บข้อมูลใน memory แทน database (ConcurrentHashMap ปลอดภัยเวลาหลาย thread ใช้พร้อมกัน)
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // ใส่ข้อมูลตัวอย่าง 3 รายการตอนเริ่มโปรแกรม
    public ProductRepository() {
        store.put("1", new Product("1", "iPhone 15 Pro (673380594-9 SEC 3)",
            "Electronics", "Apple", 50, 39900.0, "MEMBER"));
        store.put("2", new Product("2", "MacBook Air M3",
            "Electronics", "Apple", 20, 49900.0, "NONE"));
        store.put("3", new Product("3", "Samsung Galaxy S24",
            "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    // หา product จาก id: เจอ Mono ที่มีค่า, ไม่เจอ (null) → Mono ว่าง
    public Mono<Product> findById(String id) {
        return Mono.justOrEmpty(store.get(id));
    }

    // แปลงทุกตัวใน store เป็น Flux เพื่อส่งออกไปทีละตัว
    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }

    // บันทึกลง store แล้วคืน product ตัวนั้นกลับไป
    public Mono<Product> save(Product product) {
        store.put(product.getId(), product);
        return Mono.just(product);
    }

    // ลบออกจาก store แล้วคืน Mono ว่าง Mono<Void> บอกว่าลบเสร็จ
    public Mono<Void> deleteById(String id) {
        store.remove(id);
    }
}
```

```

        return Mono.empty();
    }

    // เอาทั้งหมดมากรองเหลือเฉพาะ category ที่ตรงกัน ไม่สนตัวพิมพ์ใหญ่เล็ก
    public Flux<Product> findByCategory(String category) {
        return findAll().filter(p -> category.equalsIgnoreCase(p.getCategory()));
    }
}

```

2.ProductController.java

```

Java
@RestController
@RequestMapping("/products") // ทุก endpoint เริ่มต้นด้วย /products
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    // GET /products/{id} → product 1 ตัว (Mono)
    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    // GET /products → product ทั้งหมด (Flux)
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    // POST /products รับ JSON จาก body มาบันทึก แล้วคืนตัวที่บันทึกแล้ว
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }
}

```

```

// DELETE /products/{id} ลบ ไม่มีข้อมูลส่งกลับ (Mono<Void>)
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

// GET /products/category/{category} กรองตามหมวด (Flux)
@GetMapping("/{category}/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price → ราคาหลังส่วนลด (Mono<Double>)
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
}

```

3.ProductService.java

```

Java
@Service
public class ProductService {

    // รับ repository ผ่าน constructor (Constructor Injection ตามหลัก DIP)
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // หา product ถ้าไม่เจอให้เปลี่ยน Mono ว่างเป็น error แทน
    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
    }
}

```

```

// ดึงทั้งหมดจาก repository
public Flux<Product> getAll() {
    return repository.findAll();
}

// ถ้ายังไม่มี id ให้สร้าง UUID ให้ก่อน แล้วค่อยบันทึก
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(java.util.UUID.randomUUID().toString());
    }
    return repository.save(product);
}

// ลบตาม id
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

// ดึงตาม category
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// หา product ก่อน แล้วใช้ map แปลง Product ราคาหลังลด
// เรียกผ่าน getById เพื่อให้ได้ error กรณีหาไม่เจอด้วย
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id).map(Product::getDiscountedPrice);
}
}

```